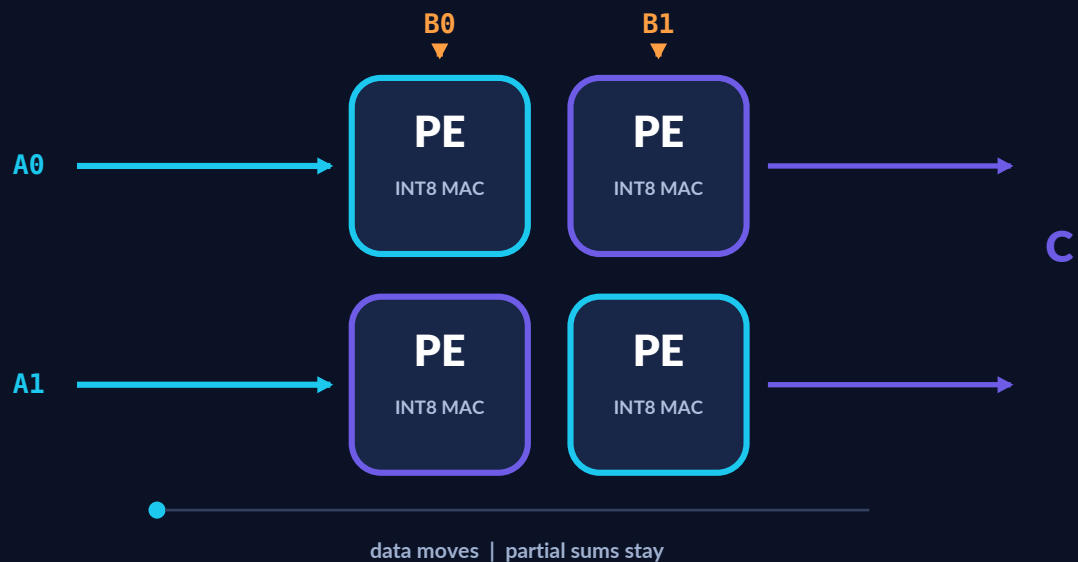


A FIRST SILICON PROJECT WITH A REAL DATAFLOW STORY

MINIMAT

A pipelined 2 x 2 INT8 systolic matrix processor



Build four processing elements. Make data move like a wave. Verify every product. Then turn the RTL into physical silicon.

4 MACs / cycle peak after fill	Signed INT8 32-bit accumulation	SPI controlled load - run - read	Open-source flow RTL to GDSII
-----------------------------------	------------------------------------	-------------------------------------	----------------------------------

01 The project in one minute

MiniMat is deliberately small in dimensions and serious in engineering depth. It is a domain-specific processor that accelerates the mathematical operation at the heart of modern neural networks.

The promise: send two small INT8 matrices over SPI, let a 2 x 2 systolic array multiply them as a pipeline, then read four exact 32-bit results back.

2 x 2

PHYSICAL PE ARRAY

K ≤ 8

FIRST-VERSION DEPTH

32 B

A + B BUFFERING

4

RESULT ACCUMULATORS

What you are building

- Four identical processing elements arranged as a systolic grid.
- A controller that launches a correctly skewed operand wavefront.
- Small local buffers so SPI loading is separated from computation.
- A command protocol for load, start, status, result, and clear.
- A self-checking SystemVerilog environment and a clean GDSII layout.

What you are not building

- Not a general-purpose CPU or a complete neural-processing unit.
- No AXI, DMA, cache, floating point, sparsity, or operating system.
- No on-chip model storage or automatic large-matrix tiler in version one.
- No vague claim of 'running AI' - the deliverable is a precise matrix engine.

Why this scope works

It is compact enough to finish, yet it forces you to solve real architecture problems: signed arithmetic, data alignment, pipeline validity, deterministic control, protocol behavior, constrained-random verification, critical-path analysis, and physical timing closure.

02 Why matrix multiplication matters

Neural networks repeatedly turn inputs and weights into dot products. Matrix multiplication is the organized way to compute many dot products at once.

A (2 x K)

1-23

40-1

X

B (K x 2)

21

-13

2-2

=

C (2 x 2)

10-11

66

Example with K = 3

Where the operation appears

AI operation	Matrix interpretation	What MiniMat demonstrates
Fully connected layer	A batch of activation vectors multiplied by a weight matrix.	Two input samples and two output neurons per hardware tile.
Convolution	Image windows are rearranged into vectors; filters become matrix columns.	The dot-product machinery, while host software handles window extraction.
Attention	Query-key and attention-value products are matrix multiplications.	The same multiply-accumulate dataflow, not a complete transformer.

The intellectually honest claim is powerful: MiniMat is a reusable INT8 matrix-multiplication tile - the computational building block from which larger AI accelerators are assembled.

Why INT8?

Quantized inference represents activations and weights with signed 8-bit integers. Compared with floating point, multipliers are smaller and data movement is lighter. Products require 16 bits, while repeated accumulation needs a wider register; MiniMat uses 32 bits to keep the first version safe and simple.

03 The exact mathematical contract

Before RTL, define the computation without ambiguity. The architecture should implement this contract bit-for-bit for every legal input.

$$\begin{array}{c} \text{A (2 x K)} \\ \begin{bmatrix} 1 & -2 & 3 \\ 4 & 0 & -1 \end{bmatrix} \end{array} \times \begin{array}{c} \text{B (K x 2)} \\ \begin{bmatrix} 2 & 1 \\ -1 & 3 \\ 2 & -2 \end{bmatrix} \end{array} = \begin{array}{c} \text{C (2 x 2)} \\ \begin{bmatrix} 10 & -11 \\ 6 & 6 \end{bmatrix} \end{array}$$

Example with $K = 3$

For each output

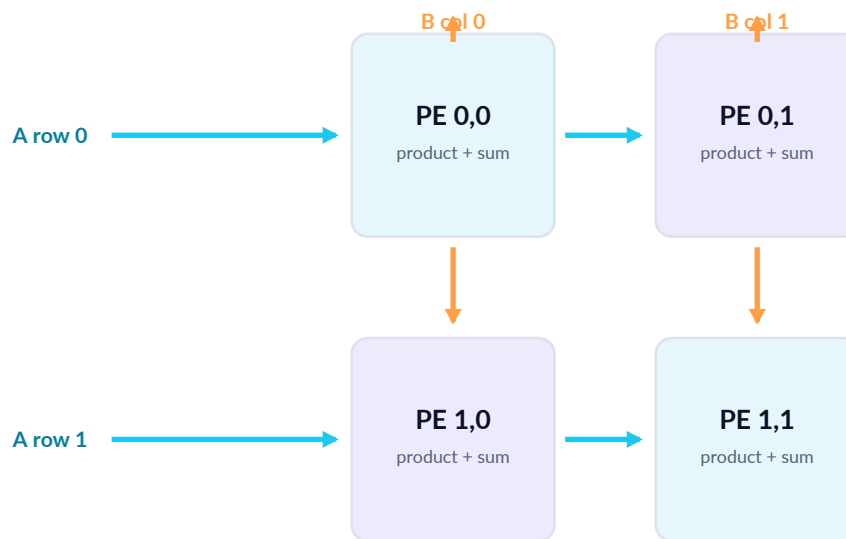
```
C[i][j] = SUM over k=0..K-1 of signed(A[i][k]) x signed(B[k][j])
```

Rule	Version-one decision	Reason
Inputs	Two's-complement signed INT8	AI inference commonly uses quantized signed operands.
Product	Signed 16-bit	The full product of two INT8 values fits exactly.
Accumulation	Signed 32-bit, wrap on overflow	Easy to specify and verify; overflow is practically unreachable for $K \leq 8$.
Output	Return all four 32-bit sums	No information is destroyed by early saturation.
K	Programmable from 1 through 8	Enough to exercise the wavefront while keeping buffers small.

Golden rule: explicitly cast both operands as signed before multiplication. A single unsigned promotion can produce plausible-looking but incorrect silicon.

04 The systolic idea

A systolic array behaves like a tiny rhythm machine: values enter on the edges, move one processing element per cycle, and meet at predictable places and times.



A values march right. B values march down. Each PE keeps its own C partial sum.

Output-stationary dataflow

Each PE owns one output $C[i][j]$. It receives a sequence of $A[i][k]$ values from the left and $B[k][j]$ values from above. The PE multiplies matching pairs and adds each product to its local accumulator. A and B continue moving; the partial sum does not.

Why it is attractive

- Four products can be formed in parallel once the wavefront fills.
- Nearest-neighbor wiring is regular and physically friendly.
- Operands are reused as they pass through multiple PEs.
- The array scales conceptually from 2×2 to much larger grids.

What makes it tricky

- Rows and columns must be skewed so matching k values meet.
- Valid bits must follow exactly the same pipeline as their data.
- Completion includes array fill, compute, and drain latency.
- A one-cycle error can corrupt only some outputs, making bugs subtle.

05 Data moves as a wave

The array is spatially pipelined. PE(0,0) begins first; PE(1,1) begins later because both operands travel farther.

PE	t0	t1	t2	t3	t4	t5
PE 0,0	k0	k1	k2	k3	-	-
PE 0,1	-	k0	k1	k2	k3	-
PE 1,0	-	k0	k1	k2	k3	-
PE 1,1	-	-	k0	k1	k2	k3

Conceptual wavefront for $K = 4$. Product-pipeline latency shifts completion, not pairing.

How to read the chart

The label k2 means that PE is processing the operand pair associated with inner-product index 2. Dashes are bubbles. With correct input skew, a diagonal wave of equal-k work moves through the array without mixing operand identities.

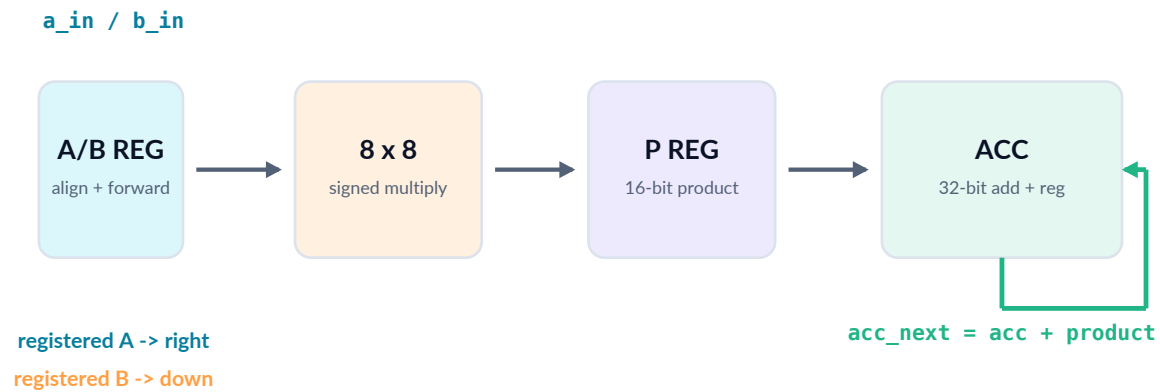
Latency versus throughput

Quantity	Meaning	MiniMat implication
Latency	Cycles from START until all outputs are captured.	Approximately K plus array fill/drain and PE pipeline delay.
Throughput	Useful MAC work completed per cycle once full.	Up to four MAC operations per cycle in the 2×2 array.
Utilization	Fraction of PEs doing useful work.	Short K spends a larger fraction filling and draining the array.

Do not hard-code a guessed DONE cycle. Derive completion from a valid token that travels through the real pipeline, then verify the cycle count independently.

06 Inside one processing element

The PE is the smallest block with the biggest consequences. Its latency defines array timing; its multiplier and adder shape area, power, and maximum frequency.



The valid bit follows the same latency as the product. Never infer validity from data values.

Signal/state	Width	Purpose
a_in, b_in	8 bits signed	Operands entering from the west and north.
a_out, b_out	8 bits signed	Registered operands forwarded east and south.
product_q	16 bits signed	Pipeline register that breaks the multiplier-to-adder path.
acc_q	32 bits signed	Output-stationary partial sum.
valid_q	Pipeline matched	Indicates that product_q represents real work.

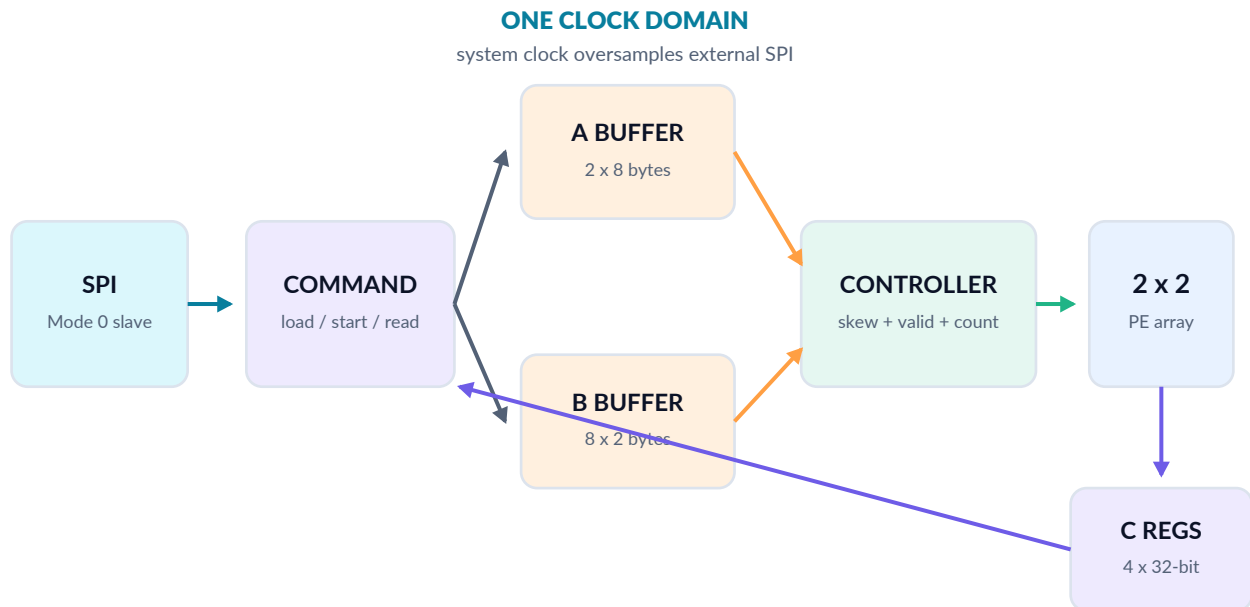
The central microarchitecture experiment

Build an unpipelined PE and a product-registered PE. The unpipelined version has lower single-operation latency and fewer registers, but the multiplier-plus-adder path is longer. The pipelined version costs registers and an extra cycle, yet should close at a faster clock. Measure this after synthesis and routing.

Pipeline registers improve frequency only when control and valid alignment are redesigned with them. Adding a register to data alone creates a faster wrong answer.

07 The complete version-one architecture

The surrounding logic exists to keep the array fed correctly and make it observable. Every block has a narrow, testable responsibility.



RTL block	Responsibility	Best first test
processing_element	Signed multiply, product pipeline, accumulate, forward A/B.	Random operand streams against a scalar sum.
systolic_array_2x2	Four PEs and nearest-neighbor connections.	Direct boundary driving; inspect every PE product.
systolic_controller	Clear, skew, valid, issue count, drain, done.	Cycle-accurate expected wavefront.
operand_buffer	Store 2 x K A and K x 2 B values.	All indices, overwrite, reset, boundary K.
spi_slave	Convert external serial bits into command bytes and responses.	Mode-0 timing, phase variation, aborted packets.
command_controller	Load, start, status, result, clear, error response.	Complete protocol sequences and illegal commands.

Keep the hierarchy visible in simulation even if synthesis later flattens it. Debugging PE-level behavior is far easier than staring only at four final sums.

08 SPI is the front door

SPI turns the accelerator into a complete, demonstrable chip. The host loads matrices, launches computation, checks status, and reads results without any on-chip CPU.

Opcode	Command	Payload / response
0x10	SET_K	One byte: legal values 1 through 8.
0x20	LOAD_A	Exactly 2 x K signed bytes, row-major.
0x21	LOAD_B	Exactly K x 2 signed bytes, row-major.
0x30	START	No payload. Reject if busy or operands incomplete.
0x31	STATUS	busy, done, error, A_loaded, B_loaded.
0x40	READ_C	Sixteen response bytes: four little-endian 32-bit sums.
0x50	CLEAR	Clear status, buffers, accumulators, and errors.
0x60	READ_ID	Fixed design signature and revision.

One-clock-domain choice

Synchronize and oversample SCLK, CS_n, and MOSI with the system clock. Detect synchronized SCLK edges internally. Run the system clock at least six times faster than SPI. This creates a documented speed limit but avoids asynchronous control crossing into the accelerator.

Example

With a 20 MHz system clock, begin with SPI at 1 MHz. Increase only after simulation and timing analysis confirm margin.

Protocol behavior to define

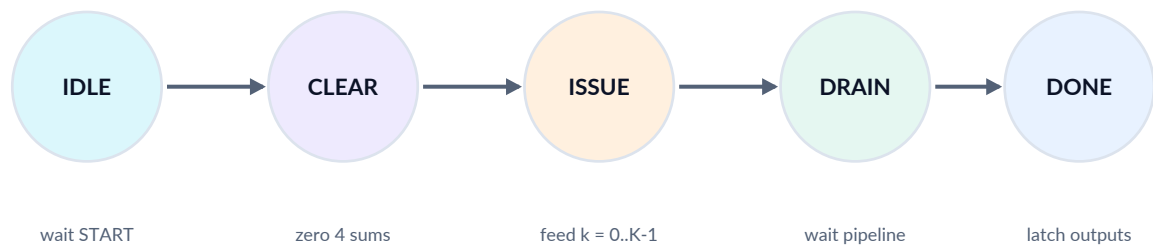
- Byte order and matrix storage order.
- What an incomplete CS-framed packet does.
- Whether LOAD overwrites or appends.
- Behavior of START while busy.
- Whether DONE is a pulse or sticky status.
- What data MISO returns during malformed commands.

A protocol is not complete until error behavior is specified. Deterministic rejection is far easier to verify and debug than 'undefined'.

09 Buffers, control, and the definition of done

The controller does not perform arithmetic. Its job is to make the right arithmetic happen on the right cycles and to expose a trustworthy completion event.

A tiny controller, but a strict contract



busy rises in CLEAR and falls only after results are captured. done is sticky until clear/new start.

Controller invariants

- K is latched at START and cannot change while busy.
- All four accumulators clear before the first valid product.
- Exactly K valid products reach each PE.
- Results are copied only after the final product is accumulated.
- Input buffers remain stable from START through DONE.
- A new job cannot destroy unread results without an explicit action.

State worth exposing

- busy and sticky done bits.
- latched K and internal issue counter.
- A_loaded and B_loaded flags.
- illegal_command and start_rejected flags.
- Optional debug: current FSM state and valid wavefront.

Why small buffers are enough

SPI loads operands before START, so the array never waits for serial traffic. This removes flow-control complexity from the compute path. The cost is only 32 bytes for $K_{MAX}=8$, implemented as flip-flops or an inferred small memory depending on the target flow.

10 What MiniMat can actually do

The physical array always produces a 2 x 2 output tile. Software can reuse that tile to build larger operations without making version-one hardware more complicated.

Use case	How it maps	What remains outside the chip
Four dot products	Each PE computes one dot product of length K.	Preparing the operand vectors.
Tiny dense layer	Two input samples x K features multiplied by K features x two output neurons.	Bias, activation, and model orchestration in version one.
4 x 4 matrix multiply	Break C into four 2 x 2 output tiles and run MiniMat repeatedly.	Tiling loops and accumulation across longer K.
Convolution kernel	Flatten an image window and filter into a dot product.	Window extraction, padding, stride, and channel scheduling.
DSP correlation	Treat one operand as samples and the other as coefficients.	Sequence preparation and repeated-window scheduling.

A concrete demo

1. A Python program creates two INT8 activation vectors and weights for two neurons.
2. NumPy computes the reference result with 32-bit accumulation.
3. The host sends K, A, and B to the chip over SPI and issues START.
4. MiniMat generates the systolic wavefront and computes four dot products.
5. The host reads C, compares every element, and prints cycle count and pass/fail.

The chip accelerates the multiply-accumulate kernel. It does not store or execute an entire neural network - and that is perfectly respectable for a first accelerator.

11 Verification is half the project

The best portfolio result is not 'it worked on one matrix.' It is a documented argument that the design behaves correctly over its entire intended operating space.

Level	Stimulus	Checker
PE unit	Random signed operands, bubbles, clear, extremes.	Scalar running sum and cycle-aligned valid.
Array unit	Directly driven A/B boundaries with varying K.	Per-PE product trace and 2 x 2 golden matrix.
Controller	Starts, resets, boundaries, illegal sequences.	State invariants and exact valid wavefront.
SPI	Random phase, gaps, aborts, malformed commands.	Protocol monitor and expected command transactions.
Full chip	Load through SPI, compute, read through SPI.	SystemVerilog or Python matrix reference model.
Gate level	A compact regression on the synthesized/post-route netlist.	Same architectural scoreboard.

1,000+

RANDOM MATRICES

8 / 8

K VALUES HIT

4

OUTPUTS CHECKED

0

UNEXPLAINED FAILURES

SystemVerilog practice

- SPI interface with clocking block and driver tasks.
- Random matrix transaction class with legal-K constraints.
- Monitors that reconstruct commands and returned results.
- Scoreboard with an independent triple-loop golden model.
- Covergroups for K, signs, zero/extreme operands, and errors.

Assertions worth writing

- Data and valid advance together between PEs.
- No accumulation occurs when product-valid is low.
- Exactly K accumulations occur per PE per job.
- Buffers are stable while busy.
- DONE implies stable captured results.
- START while busy cannot clear active state.

12 The bugs most likely to find you

Systolic-array failures are often alignment bugs disguised as arithmetic bugs. Anticipating them is cheaper than debugging them in gate-level simulation - or in silicon.

Failure mode	Symptom	Prevention
Signedness lost	Negative products become large positive values.	Signed ports, explicit signed casts, extreme-value tests.
Valid one cycle early/late	Some C elements are wrong only for $K > 1$.	Pipeline valid beside data; assert product-valid alignment.
Incorrect skew	PE(0,0) passes while off-diagonal outputs fail.	Cycle-accurate wavefront test and per-PE tracing.
Accumulator cleared late	First product disappears or stale sums leak between jobs.	Dedicated CLEAR state before any valid issue.
DONE guessed from K	Last output is missing after adding a pipeline stage.	Use a completion token through the actual latency.
SPI sampled too slowly	Intermittent command corruption depending on phase.	Document frequency ratio and randomize phase in tests.
Width truncation	Only extreme matrices fail.	Name widths, size intermediates, lint aggressively.
Reset incompletely specified	Simulation starts clean; silicon starts unpredictably.	Reset all control/valid state; do not depend on initial blocks.

Debug from the inside out: identify the first PE and first cycle where the observed product differs from the expected product. Final C mismatches alone are too late in the causal chain.

A useful debug trace

During simulation, log cycle, controller state, issue index, each PE's A, B, product-valid, product, and accumulator. Keep this trace optional with a compile-time flag so normal regressions remain fast.

13 The physical-design story

MiniMat is small enough to understand physically and rich enough to create real tradeoffs. Its repeated multipliers, wide accumulators, and pipeline registers will leave a visible signature in timing and layout.

Physical concern	What drives it	What to inspect
Critical path	8 x 8 multiplier, 32-bit addition, controller fanout.	Pre- and post-route STA path details.
Area	Four multipliers, four accumulators, 32-byte operand storage.	Cell histogram, utilization, sequential/combinational split.
Congestion	A/B broadcast edges, PE interconnect, result routing.	Global-route heat map and detours.
Clock tree	Product, operand, valid, accumulator, and buffer registers.	Skew, insertion delay, buffer count, hold fixes.
Power	Four multipliers toggling together during ISSUE.	Vectorless estimate, then activity-based estimate if available.
Pin timing	Asynchronous external SPI inputs and MISO response.	Input synchronizers and realistic I/O constraints.

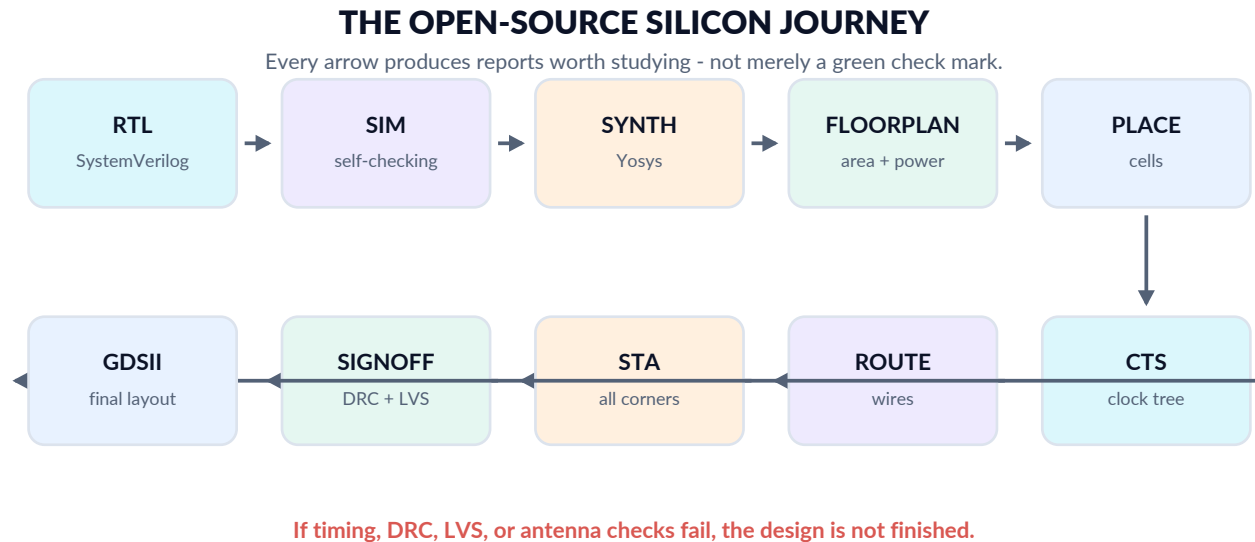
The comparison that makes the project memorable

Variant	Expected advantage	Expected cost
MAC in one cycle	Lower operation latency and fewer product registers.	Long multiplier-plus-adder timing path.
Registered product	Shorter logic depth and higher possible clock frequency.	More flops, clock power, and one additional latency stage.
K_MAX = 4	Smaller buffers and simpler address counters.	Less useful dot-product length.
K_MAX = 8	More representative AI/DSP workloads.	More storage area and switching.

Run the flow at 50 ns, 25 ns, and 20 ns clock periods. Record where timing begins to fail and explain the path - that evidence is more valuable than choosing an aggressive frequency blindly.

14 From RTL to GDSII

LibreLane orchestrates the open-source flow; Yosys and OpenROAD perform most of the synthesis and physical implementation. The goal is not only to run the tools, but to understand each report.



Milestone	Clean means
Synthesis	No unintended latches; sensible multiplier/adder mapping; area fits target.
Placement + CTS	Legal placement, controlled utilization, clock built, setup/hold repair converges.
Routing	No unresolved congestion or detailed-routing violations.
Static timing	Setup and hold pass at every required corner; I/O assumptions documented.
Physical signoff	DRC clean, LVS matches, antenna requirements satisfied.
Verification handoff	RTL regression and selected gate-level tests pass on final artifacts.

Recommended free tools

Verilator for lint/simulation/coverage; Surfer or GTKWave for waveforms; Yosys for synthesis; LibreLane and OpenROAD for implementation and STA; KLayout or Magic for layout viewing and DRC; Netgen for LVS; and the open PDK selected by the target Tiny Tapeout shuttle.

15 A ten-week build plan

Each week ends with a testable artifact. Do not integrate unverified blocks and hope the full-chip testbench will explain the failure.

Week	Focus	Exit condition
1	Freeze specification	Math contract, widths, K range, PE latency, SPI commands, error behavior.
2	Build one PE	Random unit test passes; signed extremes and bubbles verified.
3	Connect 2 x 2 array	Direct boundary test proves data propagation and four accumulators.
4	Build skew/controller	Cycle-accurate wavefront and DONE timing pass for every K.
5	Add buffers	Repeated jobs, buffer locking, and reset behavior pass.
6	Integrate SPI	Full load-start-read path works under randomized SPI phase.
7	Close verification	1,000+ random matrices, assertions, coverage report, regression CI.
8	Explore synthesis	Pipelined/unpipelined and clock-period PPA comparison.
9	Physical implementation	Floorplan, placement, CTS, route, timing iterations.
10	Signoff + portfolio	DRC/LVS/timing clean, gate test, GDS, diagrams, demo, README.

Schedule rule: when a week's exit condition is not met, fix it before adding the next block. In hardware, unresolved uncertainty compounds quickly.

Definition of done

- The final RTL and netlist produce the same matrix results for the selected regression.
- Every required K and signed corner is represented in functional coverage.
- Final timing has non-negative setup and hold slack at required corners.
- DRC, LVS, and antenna checks are clean; all unused outputs are tied deterministically.
- The repository explains how to reproduce simulation, hardening, and the host demonstration.

16 How MiniMat grows after version one

The first chip is a seed architecture. Scaling is not 'make every number bigger'; each dimension changes bandwidth, storage, verification, and physical-design pressure.

Upgrade	New capability	New cost / problem
4 x 4 PE array	Sixteen peak MACs per cycle and larger native output tiles.	Roughly 4x PEs; much greater input bandwidth, area, clock load, and congestion.
Larger K buffers	Longer dot products without repeated host loading.	Storage rapidly dominates a tiny shuttle design.
Double buffering	Load the next matrices while the current job computes.	Twice the buffer storage plus ownership and flow-control logic.
Bias + requantization + ReLU	Produces ready-to-use INT8 neural-layer outputs.	Rounding, saturation, per-channel parameters, more corner cases.
AXI4-Lite wrapper	Easy integration into an FPGA/SoC as a memory-mapped peripheral.	Five-channel protocol verification and backpressure behavior.
AXI-Stream + DMA	Sustained high-bandwidth operand delivery.	Burst engines, buffering, arbitration, and system memory assumptions.
Weight stationary mode	Better reuse when one weight tile serves many activations.	More complex loading, mode control, and verification matrix.
Sparse execution	Skip zeros to reduce wasted operations.	Metadata, irregular scheduling, load imbalance, and reduced regularity.

A sensible project family

1. MiniMat v1: 2 x 2, SPI, INT8, full 32-bit results.
2. MiniMat v2: bias, rounding, saturation, and ReLU.
3. MiniMat SoC IP: AXI4-Lite control plus streaming operand interface.
4. MiniMat-X: 4 x 4 array with double buffering and systematic PPA scaling study.
5. Future processor: a small scalar control core that schedules one or more MiniMat tiles.

17 Turn the project into a compelling engineering story

Recruiters are impressed by decisions, evidence, and finished artifacts. The GDS image is the hook; your reasoning is the substance.

Portfolio package

- One-page architecture diagram and precise design specification.
- Cycle-by-cycle wavefront illustration.
- Clean RTL hierarchy and reproducible build commands.
- Verification plan, assertions, coverage, and regression dashboard.
- Pipelined versus unpipelined PPA table.
- Critical-path report and congestion screenshot.
- Final GDS view and Tiny Tapeout integration status.
- Short SPI/NumPy demonstration video.

Questions you should answer

- Why output-stationary dataflow?
- How are rows and columns skewed?
- How do you prove every PE sees matching k operands?
- Why INT8 inputs and a 32-bit accumulator?
- What changed after adding the product register?
- What is peak throughput versus job latency?
- What limited frequency after routing?
- How would 4 x 4 change bandwidth and utilization?

Your strongest sentence: 'I built and verified a pipelined output-stationary matrix engine, measured the latency-area-frequency tradeoff, and closed the design through physical signoff.'

Suggested CV bullet

Designed and verified an SPI-programmable 2 x 2 INT8 systolic matrix processor with four pipelined MAC processing elements; developed constrained-random SystemVerilog verification and completed synthesis, timing closure, physical implementation, DRC/LVS, and GDSII generation using the open-source LibreLane/OpenROAD flow.

18 Before writing the first always_ff

A short written specification will save days of waveform archaeology. Freeze these choices, create the repository structure, and make the first PE test pass.

Decision	Recommended v1 answer	Status
Input and product signedness	INT8 signed x INT8 signed -> signed 16-bit product	[]
Accumulator behavior	Signed 32-bit, wrap semantics	[]
Array geometry	Physical 2 x 2, output stationary	[]
K range	1 through 8, latched on START	[]
PE timing	Registered A/B forwarding; registered product; accumulate on valid	[]
Completion	Sticky DONE after captured final accumulators	[]
SPI	Mode 0, MSB first, system-clock oversampled	[]
Protocol errors	Sticky error bits and deterministic command rejection	[]
Reset	Synchronous active-low internally after input synchronization	[]
Verification target	1,000 random matrices plus directed corner cases	[]
Initial clock	20 MHz compute clock; 1 MHz SPI	[]
Physical target	Current official Tiny Tapeout HDL template and matching PDK	[]

Your first seven days

1. Write a two-page specification containing the table above and PE cycle timing.
2. Create the RTL/test directory structure and a one-command Verilator test target.
3. Implement a scalar golden MAC model in the testbench before implementing the PE.
4. Write the PE, then test continuous valids, bubbles, clear, and signed extremes.
5. Draw the expected 2 x 2 wavefront on paper and turn it into a cycle table.
6. Commit only when the test is self-checking and repeatable from a clean checkout.

19 Reference shelf and quick glossary

Use current official documentation when selecting the shuttle and tool versions. The architecture in this guide is stable; submission constraints evolve.

Primary references

Tiny Tapeout technical specifications - Pin, clock, memory, and platform constraints.

<https://tinytapeout.com/specs/>

Tiny Tapeout memory guidance - Flip-flop storage and available memory options.

<https://tinytapeout.com/specs/memory/>

Tiny Tapeout HDL template - Repository structure, test hooks, and automated GDS build.

<https://github.com/TinyTapeout/ttihp-verilog-template>

Tiny Tapeout local hardening - Local LibreLane-based RTL and gate-level workflow.

<https://tinytapeout.com/guides/local-hardening/>

LibreLane documentation - Open-source ASIC implementation flow.

<https://librelane.readthedocs.io/en/stable/>

OpenROAD documentation - Placement, CTS, routing, optimization, and analysis engine.

<https://openroad.readthedocs.io/en/latest/>

Verilator language support - SystemVerilog, assertions, and coverage support and limitations.

<https://verilator.org/guide/latest/languages.html>

Yosys SBY documentation - Optional formal-property checking workflow.

<https://yosyshq.readthedocs.io/projects/sby/en/latest/>

Glossary

Term	Meaning
MAC	Multiply-accumulate: multiply two operands and add the product to a running sum.
PE	Processing element: one repeated compute cell inside the systolic array.
Systolic array	A regular grid where data advances between neighboring PEs on clock cycles.
Output stationary	Partial output sums remain local while activation and weight operands move.
Wavefront	The diagonal pattern of valid work moving through a spatial pipeline.
PPA	Power, performance, and area - the central physical-design tradeoff.
STA	Static timing analysis: proving setup and hold timing without dynamic test vectors.
DRC / LVS	Design-rule checking and layout-versus-schematic verification.
GDSII	The final geometric layout database delivered for chip integration/fabrication.

Build the smallest version that teaches the whole journey. Then scale from evidence.